Secure RAG Gateway: Defending an LLM Application Against Prompt Injection, Knowledge-Base Poisoning, and Excessive Agency

Project Outline – Security Engineering

1 Overview and Objectives

- Build a Retrieval-Augmented Generation (RAG) chatbot and a layered security architecture around it.
- Identify the threats specific to RAG and agentic AI systems, implement controls against each, and verify the controls with attack testing.

2 System Under Protection

- Baseline application: document store, embeddings, retrieval, local LLM (Ollama, Llama 3.1), and an audit log (SQLite).
- Assets to protect: the knowledge base, user data, the system prompt, and the agent's tool actions.
- Architecture: user → security gateway → RAG pipeline → LLM → agent tools → output check, with audit logging at every step.

3 Threat Model

- Attacker capabilities: can submit queries and can add documents to the corpus.
- Attack scenarios:
  - Direct prompt injection
  - Indirect injection through a poisoned document
  - Knowledge-base poisoning
  - Sensitive data leakage in model output
  - Unauthorized tool calls (excessive agency)
- Threats are mapped to the OWASP Top 10 for LLM Applications.
- Out of scope: attacks on the model itself and infrastructure compromise.

4 Security Controls

Threat	Control	Component
Direct prompt injection	Pattern-based input screening (classifier optional)	Security gateway
Sensitive data exposure	PII redaction on input, leak filtering on output	Security gateway
Abuse and cost attacks	Per-user rate limiting	Security gateway
Knowledge-base poisoning	Embedding outlier detection, instruction scanning, source trust scoring	Poisoning detector
Excessive agency	Tool allow-list, request-consistency check on sensitive tools	Agent guardrails
Lack of visibility	Audit logging of every decision	All components

5 Implementation

- Stack: Ollama (Llama 3.1), LangChain, ChromaDB, SQLite, FastAPI; rate limiting is in-memory (Redis as future work).
- Build order: manual RAG baseline, LangChain version, gateway, guardrails, poisoning detector, agent layer.
- Deployment: Docker Compose. Local Kubernetes (`kind`) with NetworkPolicies is planned as an extension. Everything runs locally at no cost.

6 Security Testing and Evaluation

- An attack suite and a benign suite, each run with controls off and then on.
- Metrics: detection rate, false-positive rate, and latency overhead.
- Demonstrations: each attack succeeds without controls and is blocked with them.

7 Results and Residual Risk

What the controls caught, what got through and why, and known limitations.

8 Conclusion and Future Work

- Conclusion: summarize which controls were effective, which attacks evaded them, and what the results show about securing RAG and agentic systems with layered defenses.
- Future work:
  - Replace the in-memory rate limiter with Redis for multi-instance deployment.
  - Move storage to PostgreSQL with pgvector for production use.
  - Add a trained classifier for injection detection to resist rephrased attacks.

- Test against adaptive attackers who know the defenses.
- Extend the agent guardrails to more tools and to multi-step agent plans.
- Deploy on Kubernetes with NetworkPolicies for network-level isolation.

References

[1] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. `https://owasp.org/www-project-top-10-for-large-language-model-applications/`

[2] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*.

[3] Greshake, K., et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proc. 16th ACM Workshop on Artificial Intelligence and Security*.

[4] Zou, W., et al. (2025). PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. *USENIX Security Symposium*.